



File System Implementation

Ivy Peng
Associate Professor
EECS, KTH



Outline

- **Layered File System Structures**
- On-disk Structure v.s In-memory Structures
- File Operations
- Allocation Methods
- Memory Mapped Files

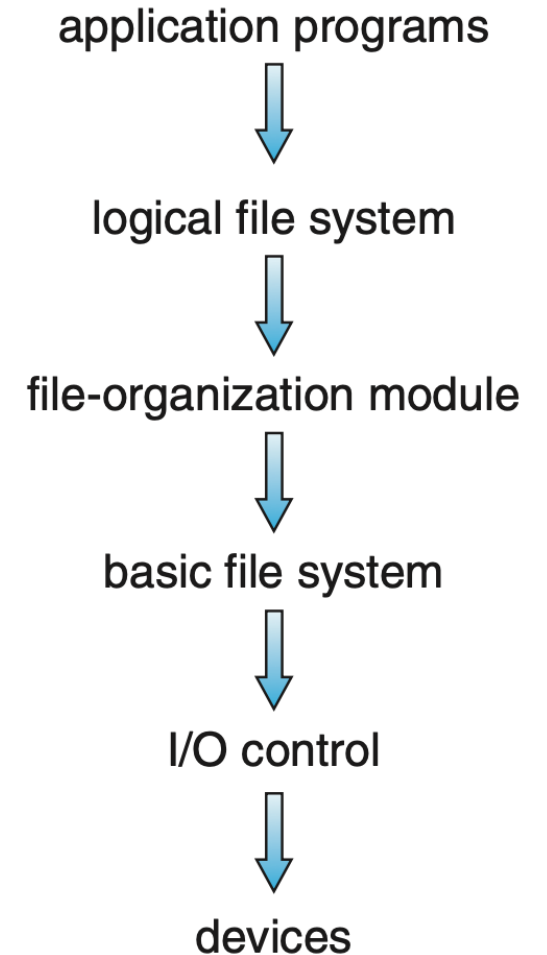


File System Tasks

- Need to
 - organize files
 - allocate disk space
 - recover free space
 - track the locations of data
 - interface other parts of the OS to secondary storage

Layered File System Structure (1/2)

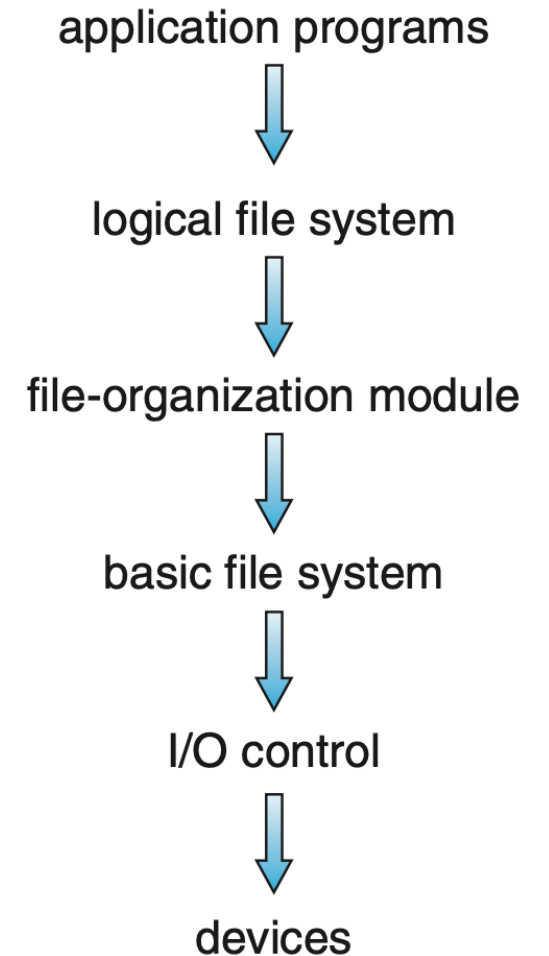
- Logical File System manages metadata, including
 - the directory structure : for file organization
 - the file structure via file-control blocks (**FCB**): ownership, permissions, location, etc
- File Organization module tracks each file's logical blocks and a free-space manager



Layered file system

Layered File System Structure (2/2)

- The basic file system layer
 - issue commands to device driver
 - manages memory buffer/caches
 - performs I/O request scheduling
- The I/O control level consists of device drivers and interrupt handlers
 - A device driver translates a high-level command into hardware instructions understandable by I/O controller



Layered file system



File System Implementation

- File systems are mostly based on-disk structures and in-memory structures
- On-disk structures
 - Boot control block (per volume): for booting an OS
 - Volume control block (per volume): #of blocks, size of blocks, free blocks...
 - Directory structure (per file system)
 - File control block (per file)
- In-memory structures
 - Mount table
 - Directory structure cache
 - The open-file table (system-wide and per process)
 - Buffers of the file-system blocks

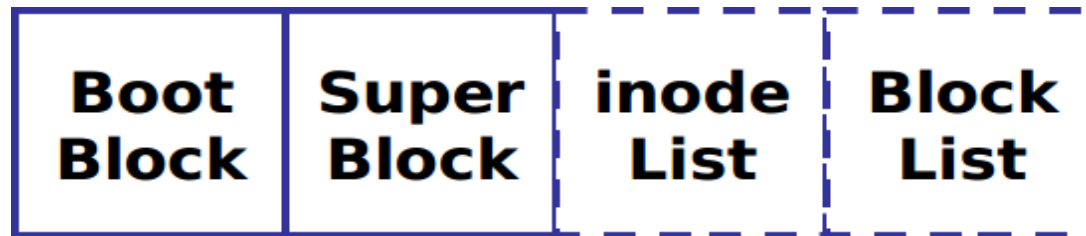


Outline

- Layered File System Structures
- **On-disk Structure v.s In-memory Structures**
- File Operations
- Allocation Methods
- Memory Mapped Files

On-Disk File System Structures (1/3)

- **Boot control block** contains information needed by system to boot OS from that volume.
 - First block(s) of the disk, empty if no OS in the volume
 - Needed if volume contains OS, usually first block of volume.
 - In Unix file system (UFS), it is called **boot block**
- **Volume control block** contains volume details.
 - Total num. of blocks, num. of free blocks, block size, free block pointers
 - In Unix file system (UFS), it is called **super block**



UFS on-disk structures

On-Disk File System Structures (2/3)

- **Inode Table/List:** A table of fixed-size inode entries.
 - Each file / directory has an inode (index node) storing metadata of the file
 - E.g. file type, size, permission, **block pointers to data blocks**, etc.
- **Block List:** a list of data blocks storing the actual data content of files
 - Each file may consist of multiple data blocks
 - Use block pointers in inode to access these blocks



UFS on-disk structures

On-Disk File System Structures (3/3)

- **Directory** structure organizes the files.
 - In UFS, this includes file names and associated inode numbers.
- File Control Block (**FCB**) contains many details about the file.
 - In UFS, inode number, permissions, size, dates.
 - Often cached in memory for performance

FCB

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks



In-Memory File System Structures

- Mount table contains information about each mounted volume.
- Directory structure cache holds the directory information of recently accessed directories.
- System-wide open-file table contains a copy of the FCB of each open file.
- Per-process open-file table contains a pointer to the appropriate entry in the system-wide open-file table.
- Buffers hold file-system blocks when they are being read from disk or written to disk.



Outline

- Layered File System Structures
- On-disk Structure v.s In-memory Structures
- **File Operations**
- Allocation Methods
- Memory Mapped Files

Check a File's Metadata

- Use command '\$ stat filename_xx'

```
ip2@ip2-QEMU-Virtual-Machine:~$ stat mmap_huge
  File: mmap_huge
  Size: 9176          Blocks: 24          IO Block: 4096   regular file
Device: fd02h/64770d Inode: 3932743       Links: 1
Access: (0775/-rwxrwxr-x)  Uid: ( 1000/   ip2)   Gid: ( 1000/   ip2)
Access: 2024-10-27 14:45:24.055468811 +0100
Modify: 2024-10-27 14:44:08.273695673 +0100
Change: 2024-10-27 14:44:47.634559062 +0100
 Birth: 2024-10-27 14:44:08.261698612 +0100
```

Question: what is the file's actual size in bytes? What is the disk usage in bytes?



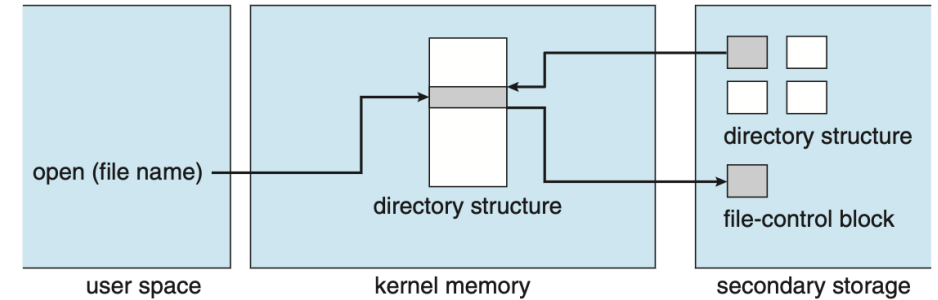
Create a File

- A program calls the logical file system.
- The logical file system knows the format of the directory structures and allocates a new FCB.
- The system reads the directory structure into memory, updates it with the new file name and FCB, and writes it back to the disk.

Questions: what on-disk file system structures are involved?

Open a File

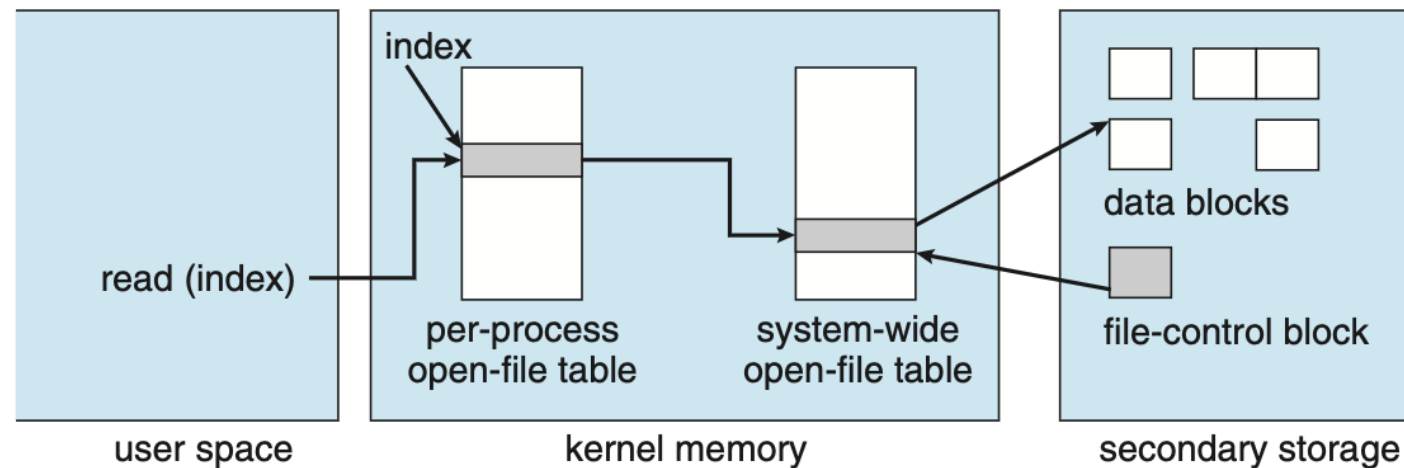
- When `open()` is called:
 - Passes 'file name' to the logical file system
 - Searches the system-wide open-file table
 - If found, a per-process open-file table entry is created
 - If not, the directory structure is searched
 - FCB is copied to the system-wide open-file table
 - An entry is added to the per-process open-file table
- Returns a pointer (file descriptor in Unix) to the entry in the per-process file-system table



Question: which structures are cached in kernel memory?

Read a File

- Only read() after the open() has returned the pointer
- All file operations are then performed via this pointer
- The pointer to the entry in the per-process file system table (with FCB cached in memory)



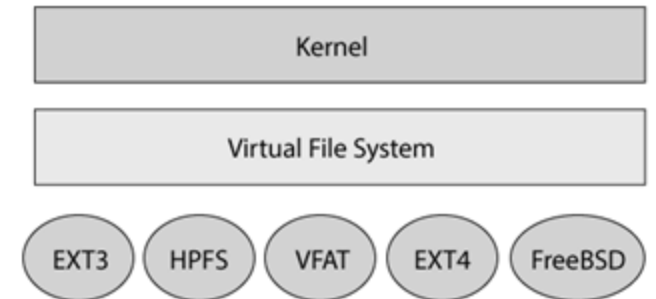
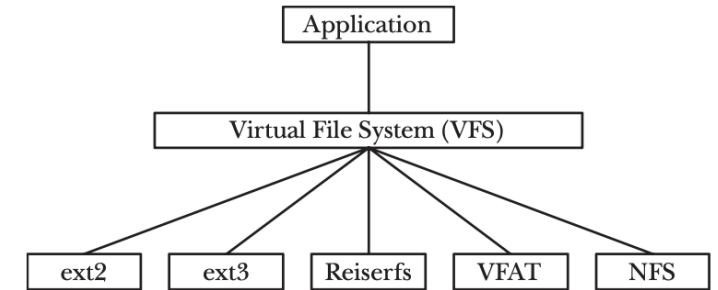


Close a File

- When a process closes the file:
 - The per-process table entry is removed.
 - The system-wide entry's open count is decremented.
- When the open count becomes 0 (all users that have opened the file have closed it), any updated metadata is copied back to the disk-based directory structure, and the system-wide open-file table entry is removed

Virtual File Systems

- VFS on Unix allows the same system call interface to be used for different file systems
 - In the last lecture, we showed that different file systems can exist on a system and mount at different points
- VFS separates file system generic operations from their specific implementation and allows access to different file systems mounted locally
- VFS is based on an abstraction called vnode (virtual node)
 - Unix inodes are unique within only a single file system, e.g., ext4
 - The kernel maintains one vnode structure for each open file
 - the underlying file system handles specific operations such as block allocation





Outline

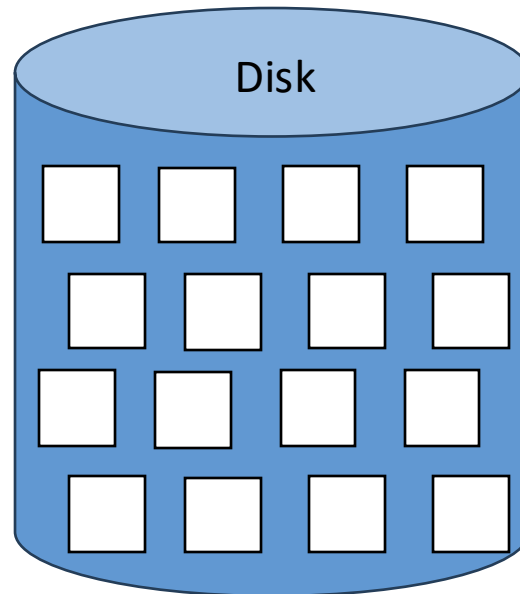
- Layered File System Structures
- On-disk Structure v.s In-memory Structures
- File Operations
- **Allocation Methods**
- Memory Mapped Files

Logical-to-physical Address Mapping

- How disk blocks are allocated to files?



Logical Address



 Blocks on Disk

Physical Address



Allocation Methods

- How disk blocks are allocated to files?
- Methods:
 - Contiguous allocation
 - Linked allocation
 - Indexed allocation

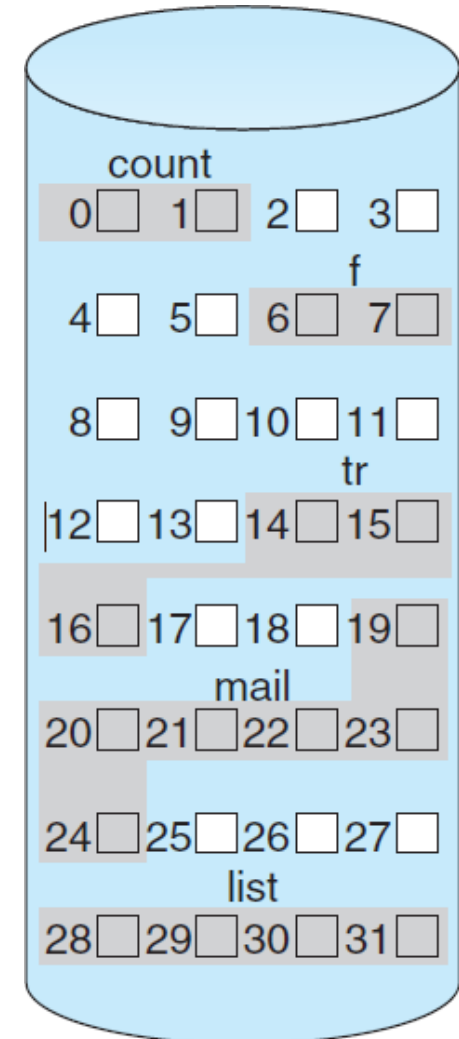
Contiguous Allocation (1/2)

- each file occupies a variable set of contiguous blocks.

directory

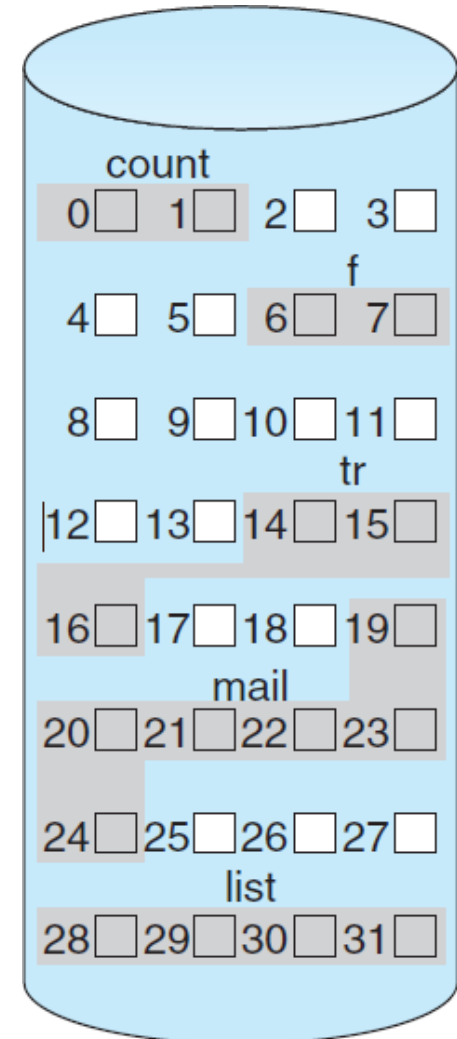
file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

- Best performance in most cases
- Simple to implement: only starting location (block number) and length (number of blocks) are need
- Supports both sequential and direct access.



Contiguous Allocation (1/2)

- Similar allocation strategies like contiguous memory allocation can be used:
- First Fit
- Best Fit
- Worst Fit
- Question: which type of fragmentation issues could occur?
 - External
 - Internal



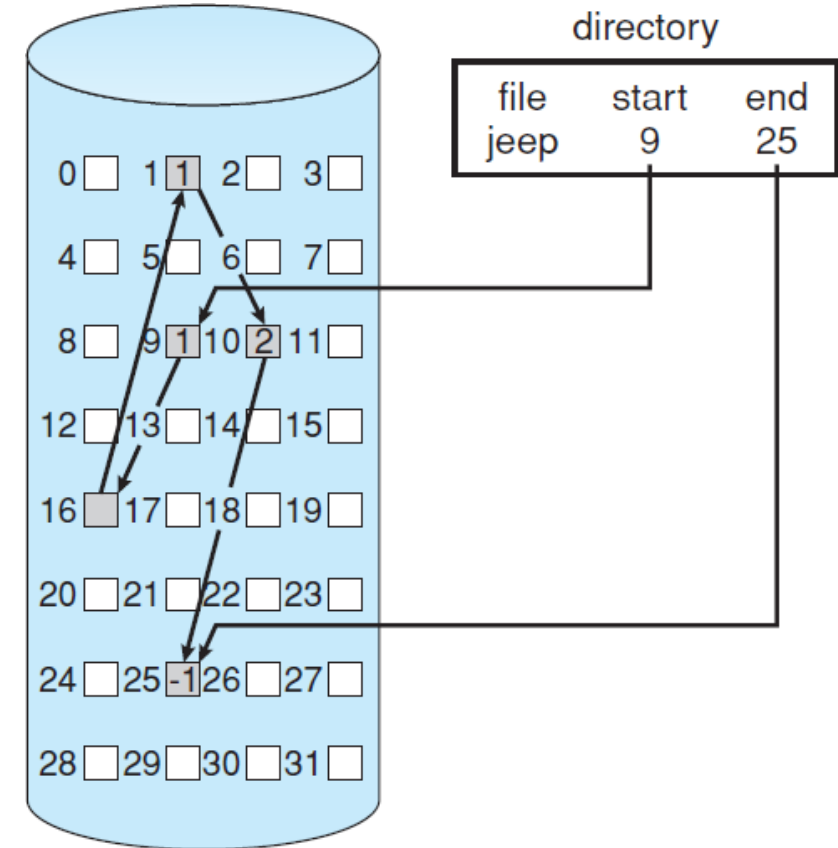


Problems in Contiguous Allocation

- Need to know file size a-prior
 - File size cannot increase after the file is created
- External Fragmentation:
 - May be resolved via **compaction**: copy an entire file system onto another device and then copy back,

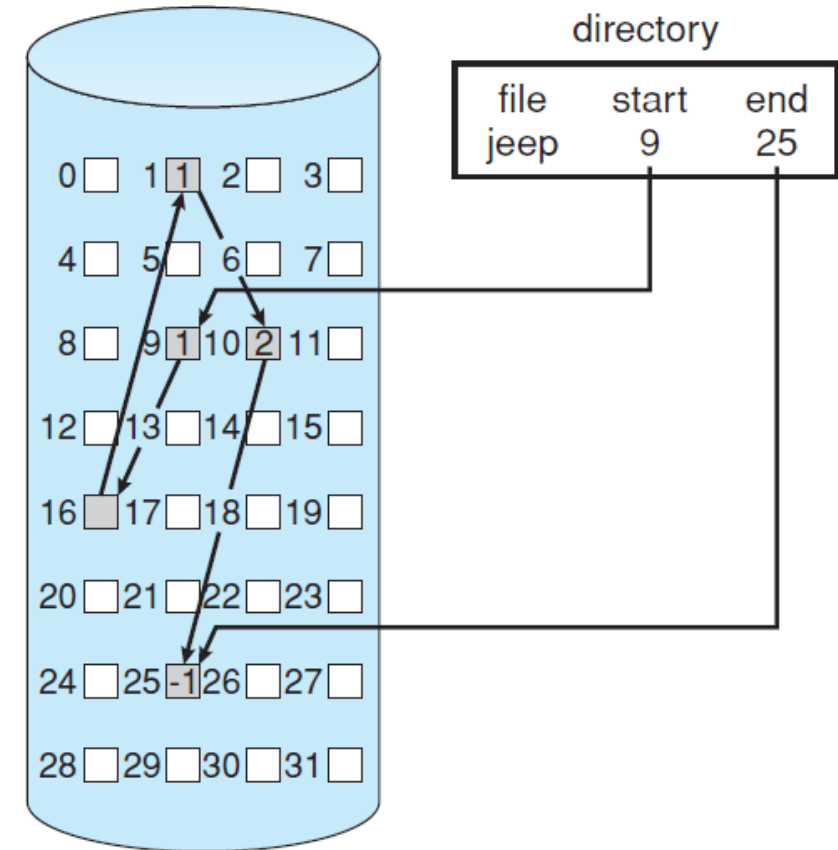
Linked Allocation

- Each file is a linked list of blocks.
- Each block contains pointer to next block.
- File ends at null pointer.
- The directory contains a pointer to the first and last blocks
- No more external fragmentation
- No compaction is needed
- A file can grow in size after creation



Linked Allocation

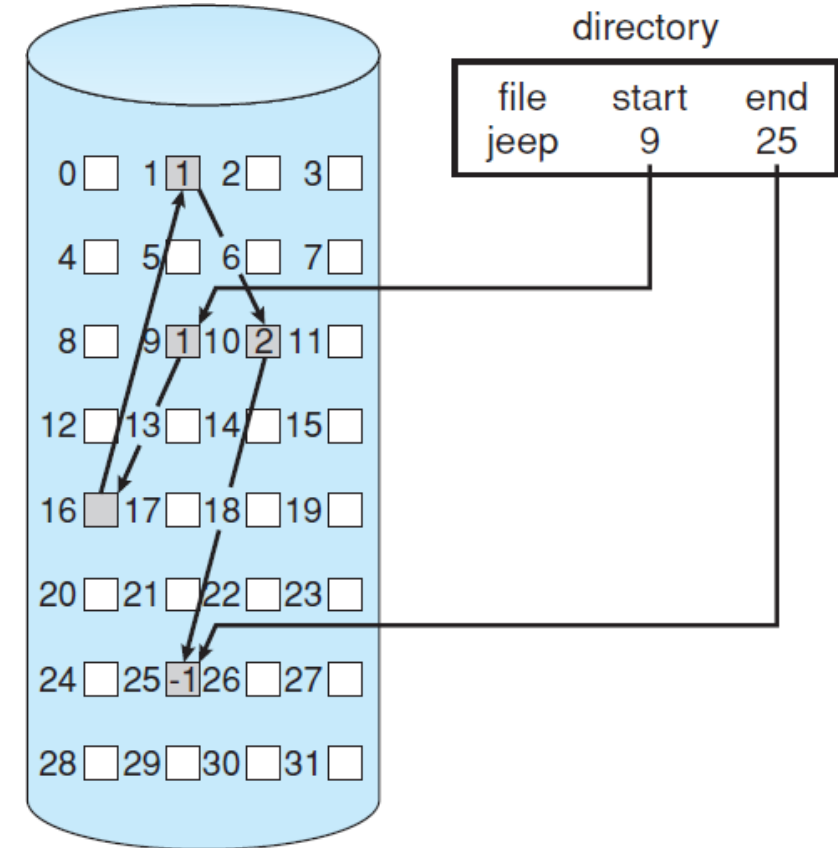
- Locating the next block needs to read in the current block from disk into memory first
- Many I/Os and disk seeks are needed to get the N-th block
- Good for sequential but not good for direct access in a file
- Reliability can be a problem
 - What if a block is defected



Linked Allocation

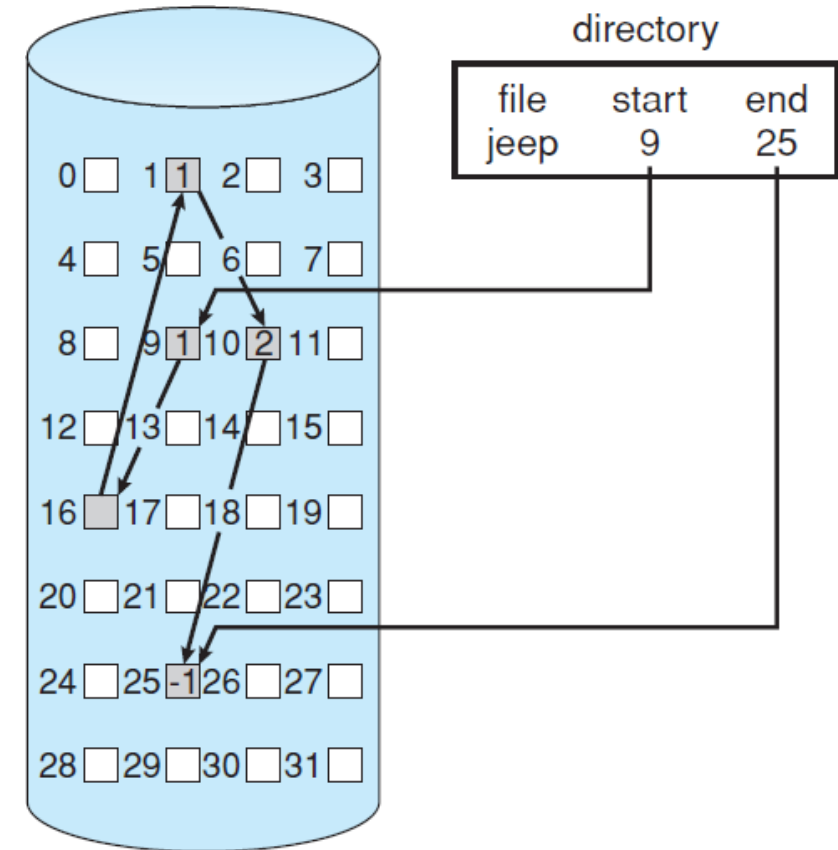
Need extra space for storing the pointers

- Each block contains a pointer to the next block.
- Assume a block is 512 bytes and a block address (the pointer) requires 4 bytes
- Space overhead for pointer is $4/512$
- Solution: clusters
 - Allocate clusters rather than blocks
 - Assume a cluster of four blocks is used.
 - Only store one pointer for every 4 blocks, i.e., Space overhead = $4 / (4 * 512)$
- However, it increases internal fragmentation



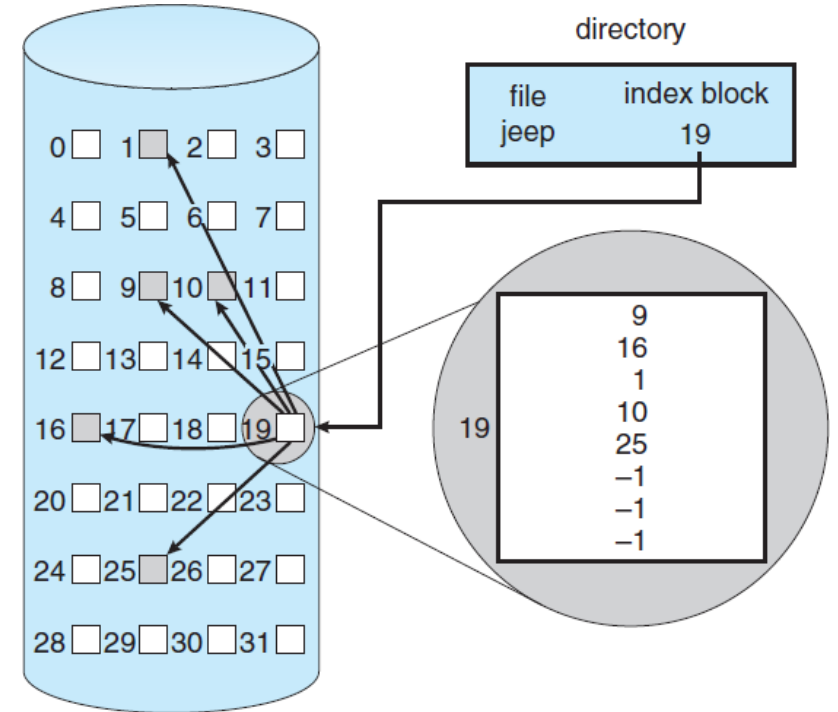
Linked Allocation

- **Free space management** system called when new block needed.



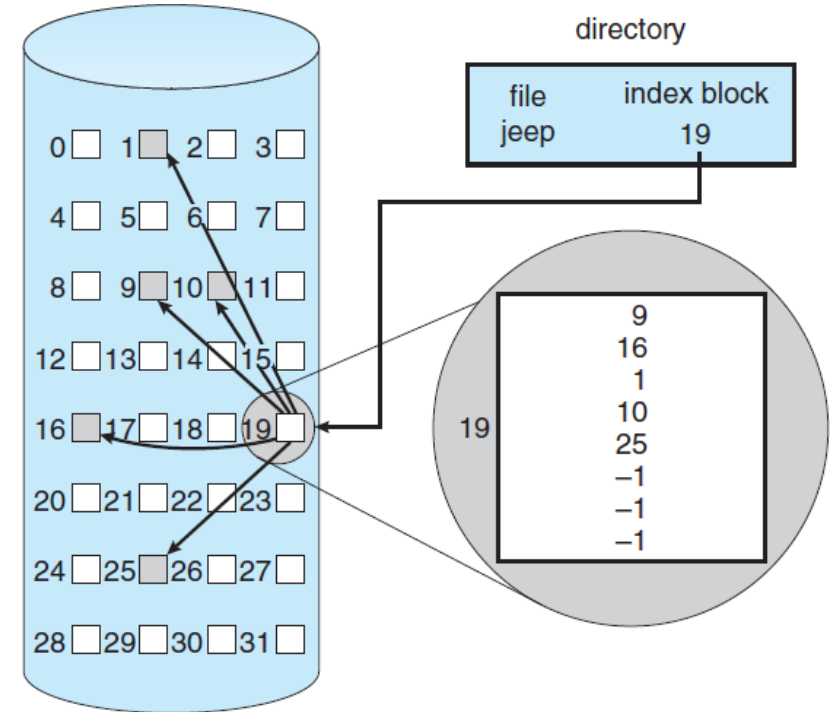
Indexed Allocation

- Each file has its own index block(s) of pointers to its data blocks.
 - E.g. block 19 is its index block in the example
- Support Random access
 - Question: which physical block should be retrieved for the 4th block in the file?



Indexed Allocation

- Dynamic access without external fragmentation
- Need index table.
- May have overhead of index block.
 - Worst scenario: even with a file of only one block, we still need an entire index block.
 - Assume block address (pointer) takes 4 bytes. A index block is 512 bytes. Space overhead = 512 / 512 = 100%
 - Suitable for large files



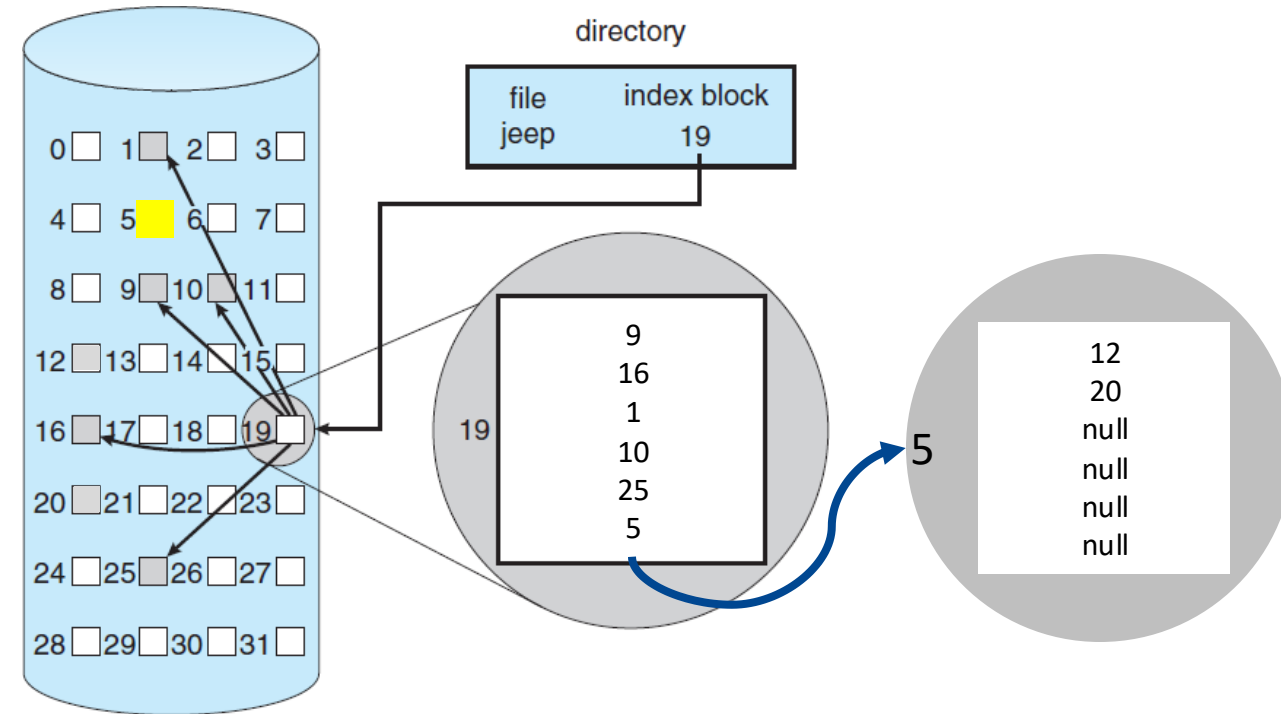


Indexed Allocation

- Need to keep the index block as small as possible
- Need a mechanism to hold pointers for large files
- Common Mechanisms for this purpose include the following:
 - Linked scheme
 - Multi-level index
 - Combined scheme

Indexed Allocation – Linked Scheme

- link blocks of index table (no limit on size)
- For example, an index block might contain a small header giving the name of the file and a set of the first 100 disk-block addresses.
- The next address is null or is a pointer to another index block.



Indexed Allocation – Multi-level Index

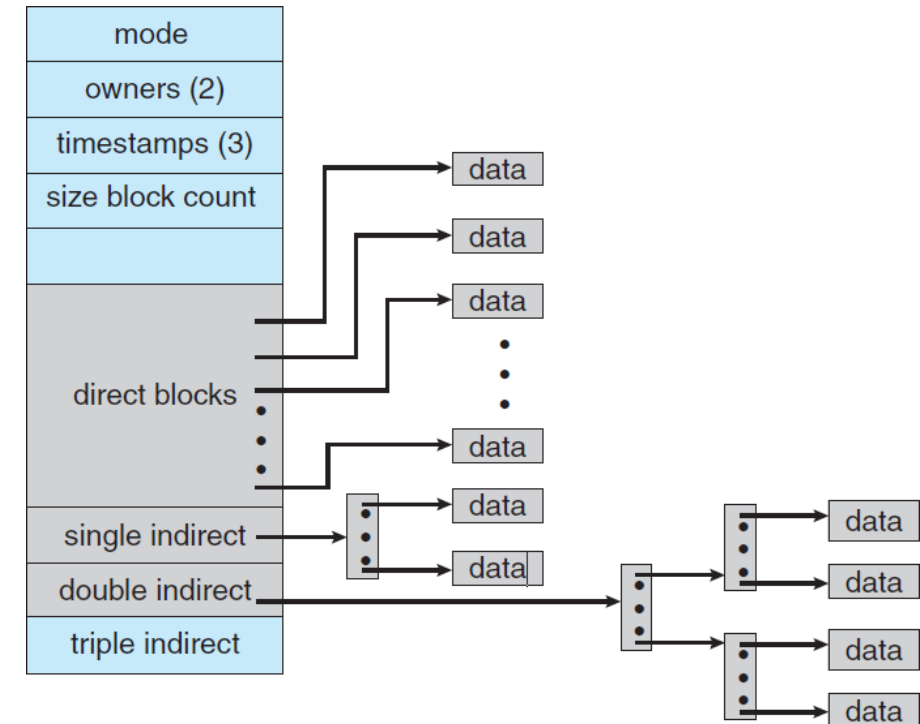
- Two-level index:
 - A first-level index block to point to a set of second-level index blocks
 - The second-level index blocks in turn point to the file blocks.
- Could be continued to a third or fourth level.
- Question:
- With 4 KB blocks and assume 4-byte pointers are used in an index block. How many data blocks and what is the maximum file size can be supported?

Indexed Allocation – Multi-level Index

- Two-level index:
 - A first-level index block to point to a set of second-level index blocks
 - The second-level index blocks in turn point to the file blocks.
- Could be continued to a third or fourth level.
- Question:
- With 4 KB blocks and assume 4-byte pointers are used in an index block. How many data blocks and what is the maximum file size can be supported?
 - Each index block can store 1024 pointers.
 - Two levels of indexes gives $1024 * 1024 = 1,048,576$ data blocks
 - Max file size = $1,048,576 * 4\text{KB} = 4\text{ GB}$

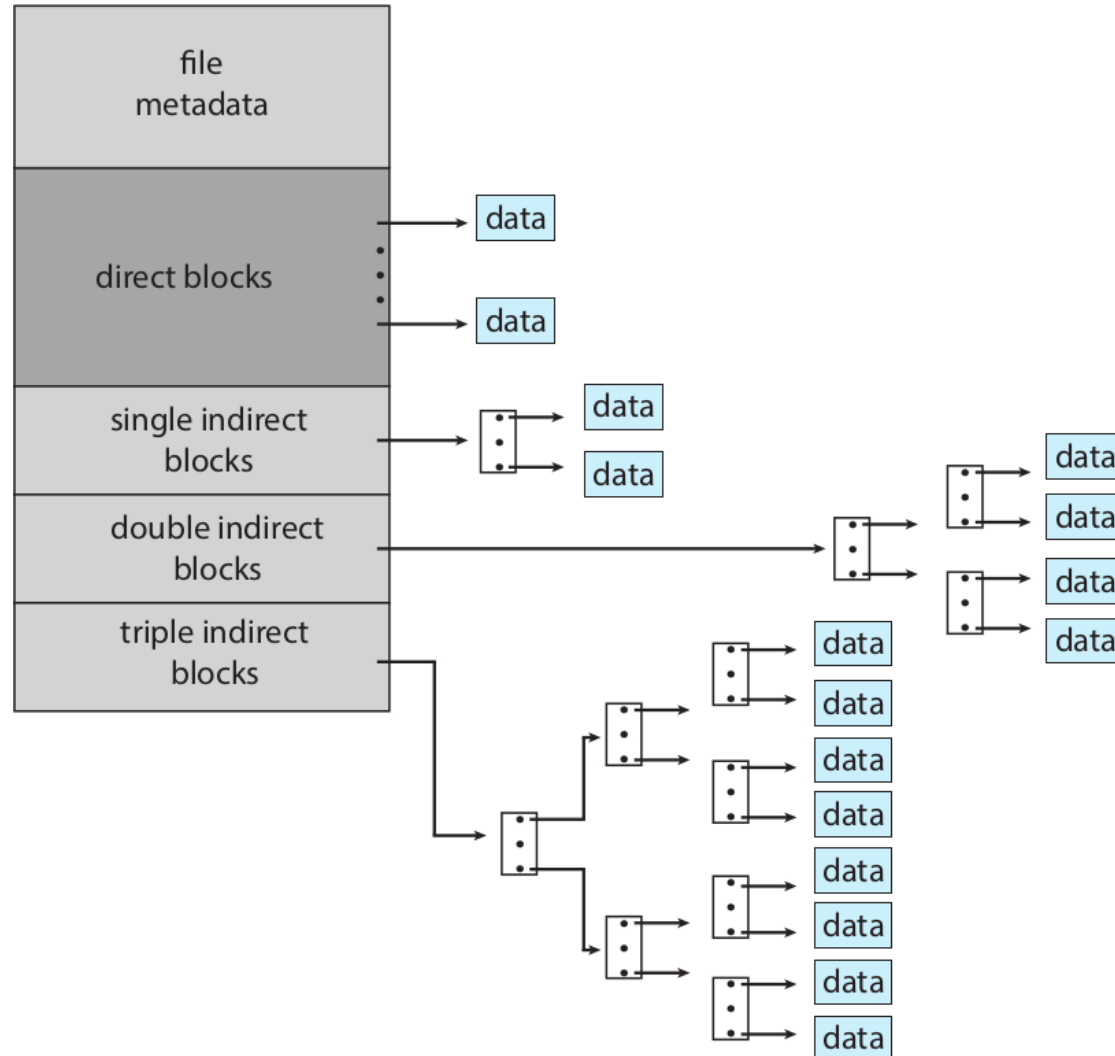
Indexed Allocation – Combined Scheme

- Combine scheme: used in Unix/Linux FS
 - The first 12 pointers point to direct blocks
 - The data for small files do not need a separate index block.
- The next 3 pointers point to indirect blocks.
 - Single indirect
 - Double indirect
 - Triple indirect



Unix Inode

- Uses the combined scheme



Free Space Management

- In Linked Allocation and Indexed Allocation, often a new block is needed to be allocated to a file
- Free space management in file systems keeps a **free-space list** recording all free device blocks not assigned to any files
 - Implementation option 1 uses a **bitmap** or **bit vector**. Each block takes 1 bit, '1' means free.

001111001111110001100000011100000 ... Question: which blocks are free?

- Implementation option 2: linked list. Useful as OS often only needs to find 1 free block.
- Implementation option 3: counting. Keep the address of the first free block and the number of free contiguous blocks. Each entry consists of a device address and a count.
- The free-space list must be modified when blocks are allocated or freed

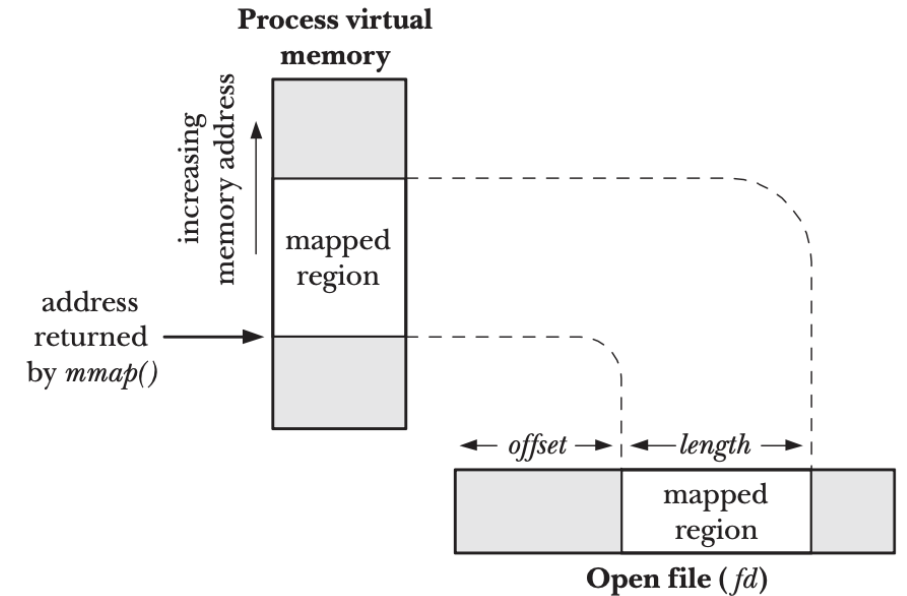


Outline

- Layered File System Structures
- On-disk Structure v.s In-memory Structures
- File Operations
- Allocation Methods
- **Memory Mapped Files**

Memory-mapped Files

- Memory mapping a file allows a part of the virtual address space to be associated with a file
- Provide a virtual memory like interface:
 - Returns a pointer to virtual address space like `malloc()`
- In contrast, each `read()`, and `write()` issues a system call and disk access



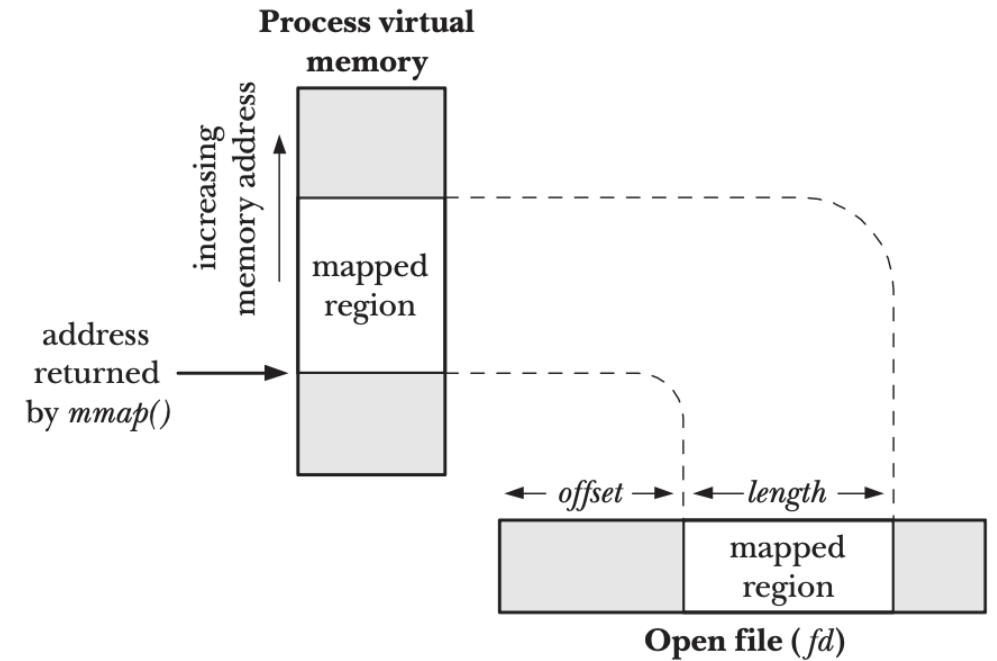
Memory-mapped Files

- Use the `mmap` system call with file backed option, by passing a valid file descriptor
 - Need to `open()` the file first
 - Support private and shared file mapping

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

Returns starting address of mapping on success, or `MAP_FAILED` on error



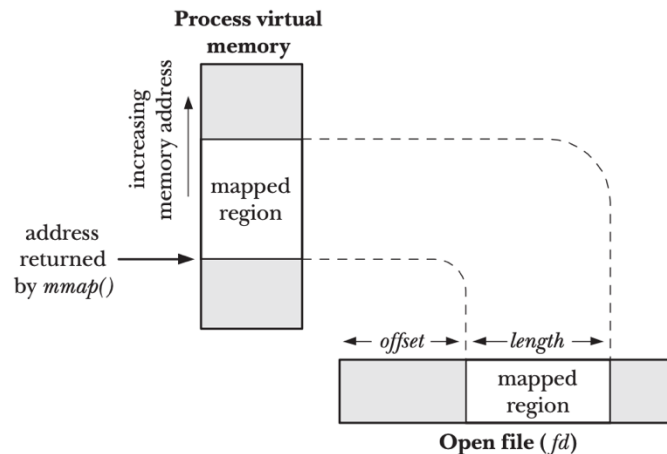
Memory-mapped Files

- Use the mmap system call with file backed option, by passing a valid file descriptor
 - Need to open() the file first
 - Support private and shared file mapping

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

Returns starting address of mapping on success, or MAP_FAILED on error



```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "tlpi_hdr.h"

int
main(int argc, char *argv[])
{
    char *addr;
    int fd;
    struct stat sb;

    if (argc != 2 || strcmp(argv[1], "--help") == 0)
        usageErr("%s file\n", argv[0]);

    fd = open(argv[1], O_RDONLY);
    if (fd == -1)
        errExit("open");

    if (fstat(fd, &sb) == -1)
        errExit("fstat");

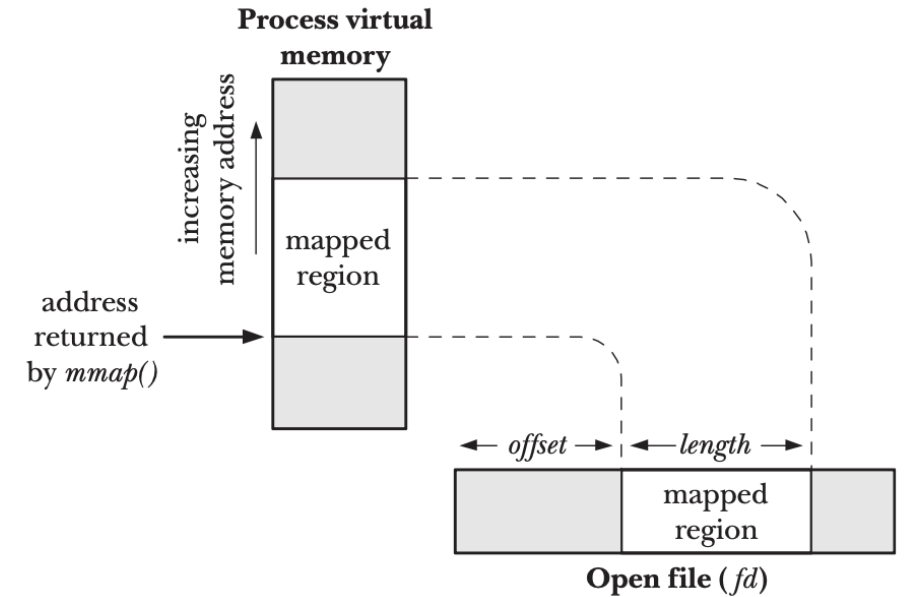
    addr = mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    if (addr == MAP_FAILED)
        errExit("mmap");

    if (write(STDOUT_FILENO, addr, sb.st_size) != sb.st_size)
        fatal("partial/failed write");
    exit(EXIT_SUCCESS);
}
```

Private File Mapping

- The *flags* argument in `mmap` : `MAP_PRIVATE`
- Modifications to the contents not carried to the underlying file
- Mostly used in the program loader and dynamic linker
 - multiple processes executing the same program or loading shared libraries

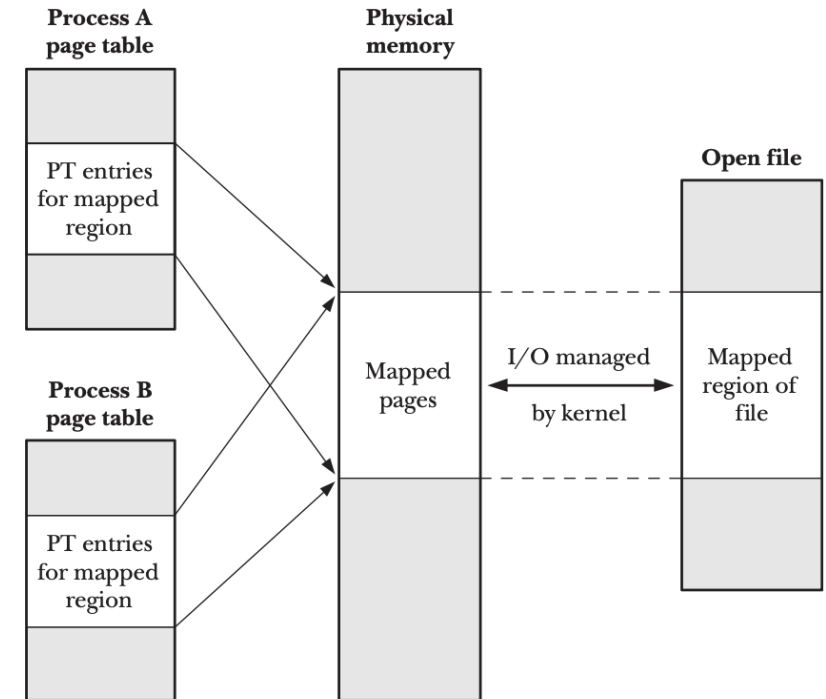
Private Mapping a file



Shared File Mapping

- The *flags* argument in `mmap` : `MAP_SHARED`
- Share the same physical pages of memory
- Modifications to the contents are visible to other processes and are carried through to the file
- However, there is no guarantee when the writes will be propagated to file
 - `msync()` system call can be explicitly used to synchronize a mapping with the underlying file

Memory Map a file into 2 process's virtual address space



Synchronize a Mapped Region

- *msync()* force data to be written to the disk and ensures the updates become visible to other processes read from the file

```
#include <sys/mman.h>

int msync(void *addr, size_t length, int flags);
```

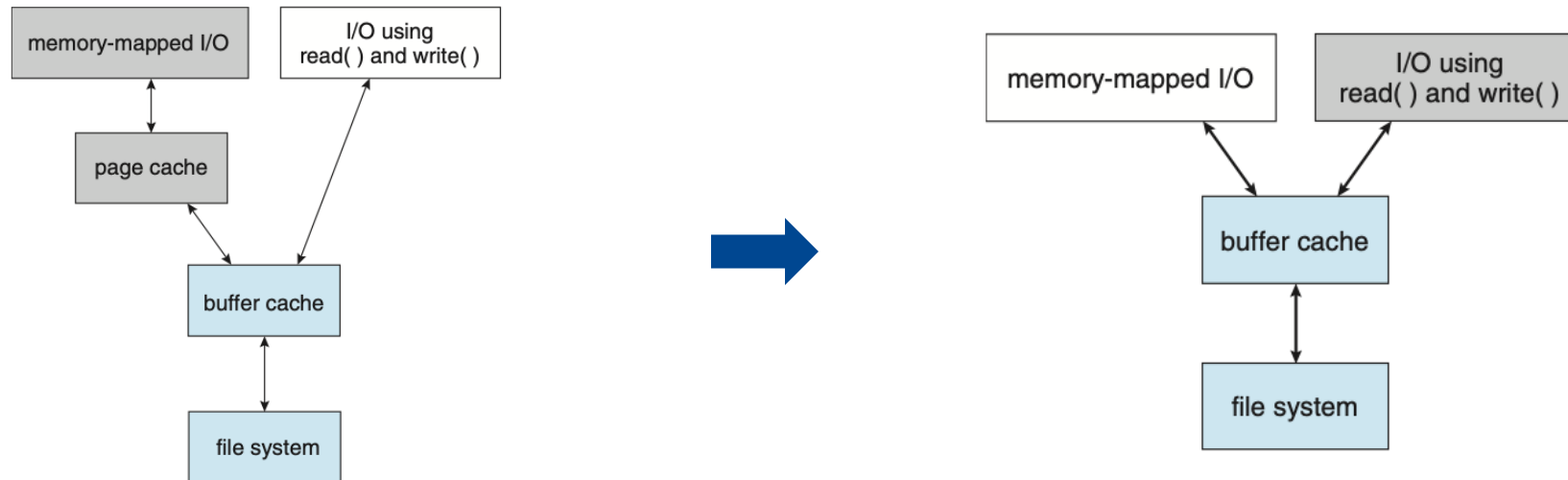
- Synchronous file write: use MS_SYNC in flags. The call blocks until all modified pages have been written into disk.
- Asynchronous file write: use MS_ASYNC in flags. The modified pages become visible to other processes and modified pages may be written to disk at a later point.

Memory-mapped I/O

- Performing file I/O by accessing bytes of memory
- The kernel ensures that the changes to memory are propagated to the mapped file
- replacing `read()` and `write()` system calls with memory accesses
- May provide better performance than conventional I/O system calls:
 - *read()* or *write()* involves two transfers: the file -> the kernel buffer cache -> the user-space buffer
 - `mmap()` eliminates the second transfer
 - Lower memory requirements: reduce the additional user-space buffer, multiple processes may share the same kernel buffer
- Mostly benefit repeated random accesses in a large file than sequential accesses
 - Question: what is the explanation.

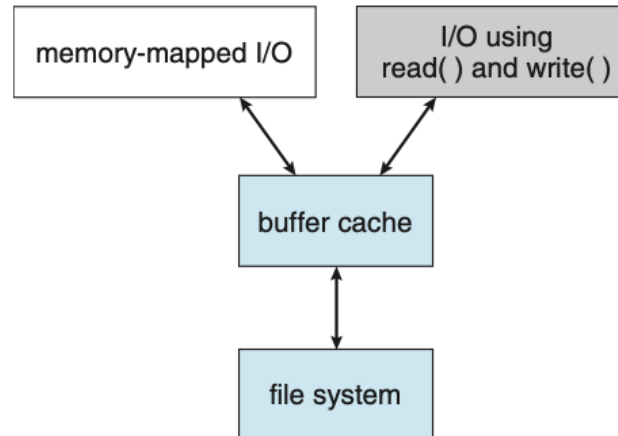
Buffer Cache

- Caching file data in main memory can boost performance
- Major OS like Solaris, Linux, and Windows, use **Page Cache** to cache both process pages and File Data
- A **unified buffer cache** is provided on Unix and Linux systems for both memory mapped I/O and direct I/O



Buffer Cache

- When process pages and page cache (for caching file data) share unused memory in a unified buffer, which system will be more impacted?



- Typically, LRU is used on the unified buffer
- Frequent I/O operations, (read and write from files) will use up most available memory
- When free memory is low, page reclamation will evict pages from process – performance impact



Summary

- Layered File System Structures
- On-disk Structure v.s In-memory Structures
- File Operations
- Allocation Methods
- Memory Mapped Files